

# AWS Well-Architected Framework

Decision-making principles

## Operational Excellence

Prioritize learning loops over static processes. Every deployment, failure, and success should inform the next decision.

Treat operations as code — not just automation, but a mindset of versioning, review, and controlled evolution.

Design for reversibility: small, incremental changes reduce risk and enable rapid course correction.

Use business-driven metrics (not technical vanity metrics) to guide operations — your observability must tell a business story, not just a system one.

Build organizational alignment first. Leadership buy-in and a defined Cloud Ops model determine whether operational excellence is possible.

Anticipate change: evolving priorities and customer needs are feedback, not disruptions. The best systems adapt gracefully.

# Security

Security is a design constraint, not a bolt-on. Every architectural choice must account for confidentiality, integrity, and availability.

Least privilege and separation of duties should shape access architecture decisions, not just IAM policies.

Favor centralized control and distributed enforcement — one source of truth, many automated enforcers.

Treat automation as your ally: use software to enforce consistency and remove human error from sensitive processes.

Visibility equals control: you can't protect what you don't monitor. Make logging and alerting a first-class design component.

Assume failure and plan for it. Incident response readiness should be validated the same way you test availability — with simulations and measurable outcomes.

# Reliability

Build for failure awareness, not failure avoidance. Expect components to break and design systems that absorb and recover.

Use redundancy and distribution to trade off between cost and resilience — a smaller number of larger resources often increases fragility.

Automatic detection and response using meaningful KPIs tied to business value, not just uptime.

Regularly test your recovery strategies — reliability is not theoretical. Chaos testing and failure injection reveal real-world weaknesses.

Manage change as the biggest source of risk. Use infrastructure as code and peer review for controlled evolution.

Balance simplicity vs. fault tolerance: every layer of redundancy adds complexity that must be justified by real resilience gains.

# Performance Efficiency

Optimize for outcomes, not technology adoption. New tech should enable measurable improvement in latency, throughput, or cost.

Use data-driven experimentation: continuously test alternative architectures and configurations to find the best-fit approach.

Favor managed abstractions when they accelerate delivery and reduce cognitive load — delegate undifferentiated heavy lifting to the cloud provider.

Balance locality vs. consistency: decide where performance matters most (e.g. latency vs synchronization).

Continuously revisit architecture fitness: today's optimal design may be tomorrow's bottleneck as patterns, demand, or customer geography evolve.

# Cost Optimization

Treat cost as a first-class architectural dimension, not as an afterthought. Every design decision should have a cost implication analysis.

Build financial visibility into every workload — map resource consumption directly to business outcomes.

Practice demand-based provisioning: scale to match usage and automate off-hours shutdowns.

Adopt ownership accountability: every workload should have a clear financial owner empowered to optimize.

Develop Cloud Financial Management as an organizational discipline — the goal is culture, not just savings.

Optimize for ROI, not reduction: sometimes higher spend is justified if it accelerates time-to-value or enables innovation.

# Sustainability

Design with resource efficiency in mind: fewer idle cycles, fewer wasteful patterns. Sustainability and cost optimization often align.

Measure impact intensity — emissions or energy per unit of useful output — and aim for continuous reduction over time.

Plan for sustainable growth: scale your architecture so that growth leads to lower per-unit impact.

Embrace shared responsibility for sustainability: we managed and services services that operate efficiently at cloud scale.

Supports innovation in efficiency: design architectures that can easily adopt newer, greener technologies as they become available.

Align sustainability with business outcomes: treat environmental performance as a measurable KPI, not a PR goal.